pdca

цикл, который доводит задачу до проверяемого результата

доменно-нейтральный контракт: PLAN → DO → CHECK → ACT
по статье `docs/ru/pdca.md`

Три проблемы вайбкодинга

Цепочка «прочитал → поправил → запустил → сказал готово» ломается в трёх местах.

- **Критерий не сформулирован заранее.** «Готово» — самооценка автора, а не проверенный факт.
- **Состояние живёт в контексте сессии.** После компакции или в новой сессии неизвестно, где мы, что сделано и почему.
- **Ошибки не наказываются.** Неудача не мешает «попробовать ещё раз» тем же способом.

Что предлагает `pdca`

- Вместо последовательности действий — **путь к проверяемому результату** (идея управленческого Plan — Do — Check — Act).
- В контракте это **машина состояний с явными выходами**: выход — условие, при котором разрешена следующая фаза.
- **PLAN** фиксирует контракт результата и критерии; **DO** выполняет и собирает доказательства; **CHECK** независимо выбирает метод и судит; **ACT** фиксирует исход.
- **Присутствие — не PASS:** работу нельзя закрыть без фактического вердикта CHECK по свидетельствам.
- Обещание скромное: цикл не гарантирует правильный результат в реальном мире, но **не даёт** закрыть работу без проверки, молча потерять состояние или выдать повторы за прогресс.

Доменно-нейтральность: когда выбирать

`pdca` называет **роли**, **фазы** и **формы** — ни артефактов, ни типов задач, ни предметных примеров.

Чем будет результат

Отчётом, документом, конфигурацией, планом — решает PLAN. Как проверять — решает CHECK.

Когда выбирать

Для задачи с **проверяемым результатом**: критерии успеха и способ их проверки можно назвать заранее.

Запуск: один вход, форма как ограничение

- Фразы «по циклу» / «через PDCA» / «запусти PDCA» ведут в **единственный вход** `pdca`.
- Его **один PLAN** выбирает форму: Bounded Execution, инкрементальная кампания (`pdca-incremental`) или коллекция (`pdca-collection`).
- Для кода тот же PLAN выбирает специализированный скилл: `pdca-coder` или `pdca-dotnet` (наследует coder).
- Явное имя другой формы или оверлея — **ограничение**: PLAN работает внутри него и не перемаршрутизирует молча.
- Прямая загрузка формы тоже грузит общие правила базы и пропускает только классификацию входа.
- Модель видит только `pdca`; остальные скиллы несут `metadata.opencode/autoinvoke: false`.

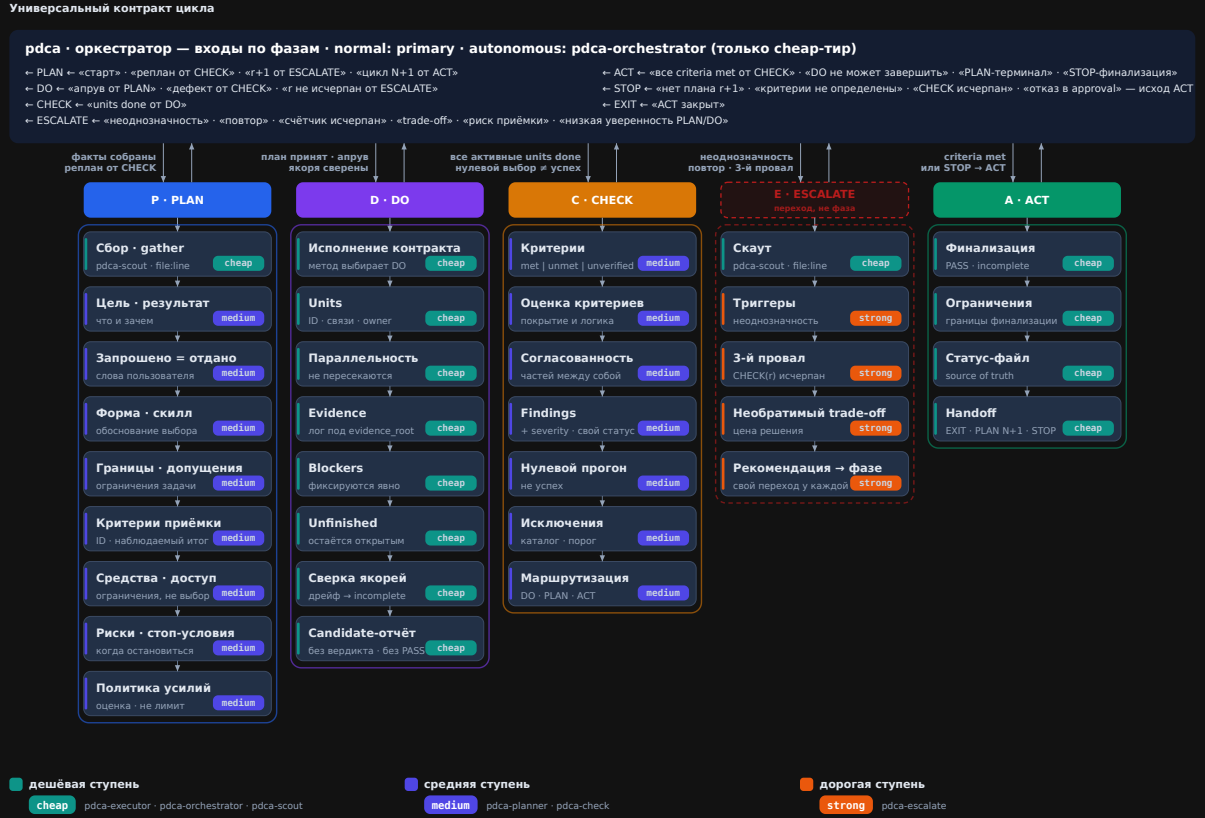
Семейство: пять скиллов, два слоя

Скилл	Что это
`pdca`	**Единственный вход** и общий контракт цикла
`pdca-incremental`	**Форма**: одна цель без известного метода/объёма, шаги на общем контракте
`pdca-collection`	**Форма**: независимые цели, каждая — полный вызов PDCA
`pdca-coder`	**Оверлей**: инженерные критерии, доказательства, команды, безопасность
`pdca-dotnet`	**Оверлей**: .NET/C# поверх `pdca-coder`

Формы **наследуют** общий контракт и добавляют только контейнер. Оверлеи **добавляют предметные факты** и никогда не открывают второй цикл.

Контракт целиком

Универсальный контракт цикла



Роли и тиры

Роль — функция с правами, а не человек и не модель. **Тир** — ярлык стоимости и качества, а **не** полномочие.

Роль	Что делает	Агент	Тир
ФАКТЫ И НАБЛЮДЕНИЕ	`file:line`, URL, числа; независимое наблюдение для CHECK	`pdca-scout`	cheap
PLAN	форма, оверлей, критерии, ревизии, терминальные исходы	`pdca-planner`	medium
DO	правит, запускает, собирает доказательства, пишет статус	`pdca-executor`	cheap
CHECK	выбирает метод для каждого критерия и выносит вердикт	`pdca-check`	medium
ESCALATE	экспертная проверка трудного или спорного решения	`pdca-escalate`	strong

Правила, которые держат роли в рабочем состоянии

- **Драйвер — диспетчер:** маршрутизирует роли и отчёты; не пишет план, не читает доказательства в контекст, не выбирает метод и не судит.
- **PLAN всегда за планировщиком:** и первый план, и каждая ревизия.
- **Писать и запускать может только исполнитель** — включая статус-файл цикла.
- **CHECK независим от автора:** судит по свежим доказательствам и может открыть артефакты доказательств цикла в пределах scoped read, но не продукт как единственное доказательство.
- **Тир — стоимость, не власть:** strong `pdca-escalate` только читает; cheap `pdca-executor` владеет правками и командами; planner/check medium, но default-deny.

Кто ведёт цикл

- **Normal mode:** primary сам ведёт цикл или шаг, просит пользователя утвердить план до DO и не вызывает `pdca-orchestrator` для этого цикла.
- **Autonomous mode:** primary делегирует координацию cheap-агенту `pdca-orchestrator`, если хост разрешает.
- Дорогой primary не запускает автономный цикл; normal разрешён в пределах прав профиля.
- Если профиль запрещает primary управлять циклом — это **заявленный блокер**, не повод расширять права.
- Нет оркестратора → **flat primary fallback** с `Notice:`; запрещённый хостом fallback не обходят.

Два режима

normal

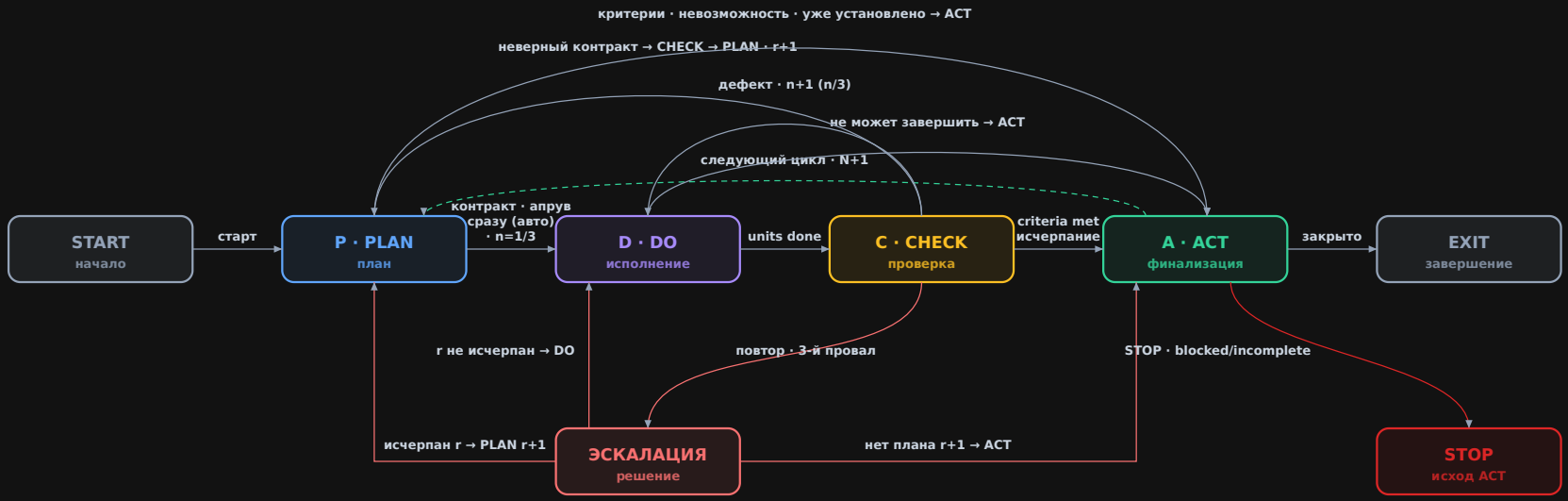
- После записи плана драйвер даёт путь к статус-файлу и просит прочитать и утвердить план.
- Явный апрув — **единственный** сигнал старта; вопрос или правка плана — не апрув.
- **Caller-driven:** исполнитель пишет durable pending ref (task/ `cycle_id` , `N` , `r` , критерии), родитель получает апрув, исполнитель фиксирует его против того же ref.
- Без апрува не создаётся и не редактируется ничего, кроме статус-файла.

autonomous

- Тот же контракт без пауз и вопросов: переход — в тот же ход, когда выполнено условие выхода.
- Ничего «для пользователя» не печатается; уведомления — в статус-файл как `Notice:` .
- Вопросов не задаётся: неразрешимое уходит в ACT как STOP.
- Апрувы, требуемые средой или пользователем, не обходятся.

Машина состояний

Граф состояний цикла



сплошная стрелка — переход

дуга — loop-back до ACT

пунктир — следующий цикл

нижний ряд — эскалация; STOP — исход ACT

PLAN: контракт результата

- цель и ожидаемый результат;
- **запрошенный результат** — объект, который просят словами пользователя, назван до критериев; план отдаёт тот же объект;
- форма и специализированный скилл с **обоснованием**: уже обоснованный метод и граница объёма либо недостающее знание;
- ограничения, допущения, средства и доступ;
- **критерии приёмки** — каждый проверяем, покрывает границу цикла;
- риски и условия остановки;
- в normal mode — оценка усилий, никогда не cutoff.

PLAN **не** выбирает реализацию и **не** выбирает метод проверки: это DO и CHECK соответственно.

PLAN: три терминальных исхода

Критерии не определены

`acceptance criteria cannot be determined`

PLAN несёт недостающую информацию и причину. DO и CHECK не запускаются.

Задача невыполнима

`original task is infeasible under the stated constraints`

PLAN несёт доказанную причину и доказательства. Один провалившийся кандидат — не доказательство.

Результат уже установлен

`requested result already established`

Факты уже определяют запрошенный результат: PLAN отдаёт его с выводом и указателями. Независимый CHECK не запускался.

Все три идут **прямо в ACT**: без DO, без CHECK, без подменного результата и без выдуманного критерия.

DO: две руки и два исхода

- Исполнитель выполняет **одобренный контракт результата** и сам выбирает реализацию внутри него.
- Работа может делиться на **юниты** с ID, критериями, зависимостями и владельцем; параллельно — только непересекающиеся.
- Каждый запуск возвращает код выхода, ключевые числа и путь к логу.
- **Исход 1:** завершённый кандидат с доказательствами → CHECK.
- **Исход 2:** «не могу завершить» с причиной, доказательствами и остаточными требованиями → ACT.
- Пути `DO → PLAN` нет.

CHECK: независимый вердикт

- Сначала оценивает **сами критерии**: покрытие, противоречия, проверяемость.
- Затем **выбирает метод** для каждого критерия и получает доказательства через драйвера.
- Статусы: `met` — выполнен, `unmet` — не выполнен, `unverified` — не проверен.
- **Отсутствие проверки — не пропуск:** без чисел, указателей или наблюдения критерий `unverified`.
- На выходе — вердикт, матрица критериев, находки, указатели и маршрут исправления.
- Совокупная оценка не компенсирует невыполненное обязательное требование.

Три выхода CHECK

CHECK → DO

Дефект реализации: критерии верны, результат им не соответствует. Исправить и проверить снова.

CHECK → PLAN

Дефект контракта: критерии противоречивы, непроверяемы или не покрывают исходные требования.

CHECK → ACT

Все критерии `met` на фактических доказательствах — либо попытки исчерпаны, и ACT фиксирует `incomplete` /STOP.

Только CHECK может вернуть работу в PLAN. DO такого маршрута не имеет.

ACT: закрытие и STOP

- ACT фиксирует то, что определила завершающая фаза: приёмку, честный `blocked` / `incomplete` /STOP или исход PLAN.
- Меняет **никакой** продукт, тест или проверенный артефакт.
- **ACT → EXIT** — после принятого CHECK: выводы, ограничения, принятый снимок, передача; статус-файл сохраняется.
- **ACT → STOP** — причины, доказательства, остаточные критерии, счётчики и история дефектов; без `PASS`, без `done`.
- STOP — **исход ACT**, а не отдельная фаза. Работа, найденная при финализации, идёт в DO и новый CHECK.

Выходы, а не нумерованные гейты

Переход	Условие
PLAN → DO	контракт полон, план в статус-файле, в normal — апрув пользователя
PLAN → ACT	критерии не определены, задача доказанно невыполнима **или** результат уже установлен
DO → CHECK	завершённый кандидат, все активные юниты закрыты на доказательствах
DO → ACT	DO не может завершить: причина, доказательства, остаток
CHECK → DO	дефект реализации или не хватает доказательств
CHECK → PLAN	дефект критериев/контракта
CHECK → ACT	приёмка на фактических доказательствах **или** исчерпание попыток

Эскалация: совещательная, не решающая

`pdca-escalate` — дополнительная экспертная проверка, не фаза и не переход.

- неоднозначное требование или критерий, где догадка недопустима;
- цикл не проходит по счётчику;
- рискованное или необратимое решение реализации;
- приёмка, требующая оценки риска;
- решение PLAN/DO, всё ещё неуверенное после целевого сбора.

Возвращает рассуждение, факты, остаточную неопределённость и рекомендованное решение. Фаза сохраняет своё решение; эксперт не отменяет CHECK, не заменяет апрув и не расширяет права.

Счётчики и история дефектов

Счётчик	Что означает	Когда растёт
`N`	номер цикла задачи (`.pdca/status/<task>-<N>.md`)	только в ACT, для следующего цикла
`r`	ревизия плана	только при настоящем изменении контракта результата/критериев
`n/3`	попытка исполнения текущей ревизии	старт 1; `CHECK → DO` +1; новая ревизия сбрасывает в 1
`rv`	ревизия контракта в пути доказательств	только когда критерии/строки меняются

- Отклонённый кандидат, уточнение, смена метода или сессии — **не** ревизия.
- Повтор того же дефекта после одного исправления — эскалация **до** следующего исправления.
- Третий проваленный содержательный CHECK одной ревизии: четвёртой попытки нет.
- `rv` — не счётчик попыток; он живёт в `.pdca/<task>/rv<k>/` .

Память и доказательства

- **Статус-файл цикла** — источник истины: живёт на диске, переживает компакцию и новую сессию, обновляется на каждом событии.
- Один писатель — `pdca-executor`; драйвер передаёт отчёты **без изменений**.
- Фиксированные разделы: `Current state`, `Original task`, `PLAN record`, `Approval record`, `DO result`, `CHECK record`, `ACT record`, `Revision history`, `Defect history`, `Changed files`, `Git state`, `Progress log`.
- **Доказательства** — `.pdca/<task>/rv<k>/`: полный лог и артефакты пишутся туда до возврата DO; CHECK читает только `.pdca/**`, поэтому указатель без файла — не доказательство.
- **Свежие доказательства:** метод выбирает CHECK, запускает исполнитель, наблюдает `pdca-scout`, судит CHECK.
- **Отчёт ≠ результат.** «Должно работать» — сигнал повторного прогона.

Восстановление после компакции

- **Источник истины — статус-файл**, не пересказ контекста.
- Детекторы: долгая сессия/авто-компакция; не можете назвать задачу, фазу, `N`, `r`, `n/3`, шаг или критерий.
- Порядок: остановить действие → прочитать статус через `pdca-scout` → перечитать контракты и red flags → восстановить состояние.
- Продолжить с текущей фазы без повторного открытия PLAN или CHECK.
- Исполнитель дописывает `Recovered after compaction`.

Пересказ по памяти — **State Rot**. Статус-файл важнее.

Инженерный оверлей: `pdca-coder`

- Не цикл, не форма и не машина состояний: добавляет инженерные факты к выбранной форме.
- PLAN формулирует наблюдаемые критерии: корректность, публичный контракт, безопасность, производительность, документацию — только где задача этого требует.
- DO выбирает реализацию и свою проверку; код, тесты и документация могут идти независимыми потоками.
- CHECK выбирает метод: диффы и антипаттерны, тесты, документация, безопасность по триггеру, производительность по количественному критерию.
- Доказательства — код выхода, числа и путь к логу; запуск с нулём целей — провал.
- Безопасность: ограниченный тестовый хост, сериализация тяжёлых наборов, потоковые логи, секреты не печатаются.

Оверлей .NET: `pdca-dotnet`

- Наследует `pdca-coder` и добавляет только .NET/C#-факты.
- **Roslyn-first:** символы, ссылки, реализации и переименования — через `roslyn`, не текстовым поиском.
- Затронутые проекты собираются последовательно; whole-solution — вне внутреннего цикла.
- `--no-build` требует артефакта, соответствующего фактическому исходнику, включая незакоммиченные изменения.
- `0 selected` — проваленный запуск; коды выхода — у настроенного раннера.
- Покрытие и мутации — числа с baseline, только когда задача или политика проекта делают их значимыми.
- Необязательные helper-скрипты: `validate_inner_loop.py` и `pdca_evidence.py`.

Коллекция: независимые цели

Коллекция — набор независимых целей, каждая — отдельный и полный вызов PDCA.

- **P:** первый PLAN каждого пункта → **ALL-барьер** → DAG/группы → DO пункта.
- **D:** линии в изолированных worktree; merge только `done`-групп, `incomplete` не сливается.
- **C:** независимый интеграционный CHECK по фактическому интегрированному снимку.
- **A:** полное завершение требует интеграционного PASS и whole-goal CHECK каждого пункта.
- Пункт может быть `cycle` или последовательной `campaign`.
- Готовность плана/подготовки — **не** одобрение и **не** завершение.
- Autocommit и `merge --no-ff` — только по проектной политике или явному запросу; push — никогда.

Кампания: одна цель без известного метода

- Один шаг = полный цикл PLAN → DO → CHECK → ACT со своим результатом и приёмкой.
- **Preparation record** — предикат, рецепт, единица инкремента, пол/цель, критерии, commit policy.
- **Baseline** снимается до DO первого шага на сопоставимой основе.
- **Принятое состояние** предыдущего шага — старт следующего; прогнозы не зачитываются.
- Один зачтённый инкремент на шаг: изменение результата **или** значимое для решения знание.
- **Граница шага:** один вызов планировщика решает и выдаёт следующий бриф или STOP.
- `met` требует whole-goal CHECK; пауза — checkpoint, а не исход.

Учебная история: где цикл останавливается

Задача: ускоряет ли кэширование промежуточных результатов ночной отчёт. Старт: `N=1`, `r=1`, `n=1`.

1. **PLAN r1** — критерии: времена с кэшем и без, вывод из чисел, воспроизводимая методика. Пользователь утверждает план.
2. **DO** — нужного набора данных нет. DO не переписывает план: возвращает «не могу завершить» с причиной и остатком.
3. **ACT** — фиксирует `blocked`, сохраняет исходные критерии и отсутствующую предпосылку. Запись закрыта, не открывается заново.
4. После новой инструкции пользователя или изменения среды начинается следующий разрешённый цикл `N+1`.

Честный исход — не провал дисциплины: цикл остановился там, где кончился проверяемый результат, и сохранил почему.

Ограничения и неверные ожидания

- «Задача слишком простая для цикла» — размер не отменяет правило приёмки.
- Дорогой primary не запускает автономный цикл; в normal он работает в пределах прав профиля.
- Недоступная роль — **resource blocker**, а не повод работать без неё.
- Полный план может быть слабым: цикл проверяет наличие и доказательства, не экспертизу.
- Отчёт ≠ результат: самопроверенный вердикт — «театр проверяющего».
- STOP — не успех и не провал: честный терминальный исход без ложного PASS.
- Статус-файл не удаляют: он финализируется и сохраняется в любом случае.

Куда дальше

- Базовая статья — `docs/ru/pdca.md`
- Полный путеводитель — `docs/ru/pdca-guide.md`
- Инженерный оверлей — `docs/ru/pdca-coder.md`
- .NET-оверлей — `docs/ru/pdca-dotnet.md`
- Коллекция — `docs/ru/pdca-collection.md`
- Инкрементальная кампания — `docs/ru/pdca-incremental.md`
- Настройка хоста — `docs/ru/install.md`

Сверено с текущими контрактами `skills/pdca*` (28.10.2026).